Programming Language Principles, Design, and Implementation

Compilers - Syntax Analysis (Parsing)

Vincent Rahli

University of Birmingham

Where are we?

- Part 1: Principles of programming
- **Part 2: Compilers**

Today

Parsing

- Context-Free Grammars
- Pushdown Automata
- Top-down parsing
- Bottom-up parsing

Further reading:

- Chapter 3 of "Modern Compiler Implementation"
- Chapter 4 of the Dragon book

Problem

- A language is a collection of strings drawn from an alphabet
- Its syntax can be specified by grammar rules

Goal: a parser processes a string of tokens, and verifies that the string can be generated from the grammar rules of the language.

In addition:

- reports **errors**
- builds an internal representation: an **abstract syntax tree** (AST)

2 main types of parsers:

- **top-down** (e.g., LL)
- **bottom-up** (e.g., LR)

Context-Free Grammars

Programming languages are typically specified using **context-free grammars** (CFGs), i.e., grammars for which the production rules can be applied regardless of the context.

Formally, a CFG can be defined as follows:

$$G = \langle N, T, P, S \rangle$$

where

- N: a set of non-terminal symbols
- T: a set of terminal symbols
- P: a set of production rules in $N \times (N \cup T)^*$
- S: a start non-terminal symbol

The terminal and non-terminal symbols are the tokens generated by the lexer.

We write $A, B, C, \ldots$ for non-terminal symbols; and $\alpha, \beta, \gamma, \ldots$ for sequences of non-terminal and terminal symbols (i.e., in $(N \cup T)^*$)

Context-Free Grammars

Example: Consider the following grammar (in BNF notation):

$$E ::= E + E \mid E * E \mid (E) \mid num$$

where $num \in \mathbb{N}$.

Its components are:

- $N = \{E\}$
- $T = \{+, *, (,), num\}$
- $P = \{(E, E + E), (E, E * E), (E, (E)), (E, num)\}$
- $S = E$

Context-Free Grammars

What are the components of the following grammar:

$$B ::= \texttt{true} \mid \texttt{false} \mid B \;\&\&\; B \mid B \;||\; B$$

Its components are:

- $N = \{B\}$
- $T = \{\texttt{true}, \texttt{false}, \&\&, ||\}$
- $P = \{(B, \texttt{true}), (B, \texttt{false}), (B, B \;\&\&\; B), (B, B \;||\; B)\}$
- $S = B$

Derivations

A **derivation** is a sequence of steps of the form

$$\alpha A \beta \Rightarrow \alpha \gamma \beta$$

such that $(A, \gamma) \in P$, i.e., it is a production rule

We write $\Rightarrow^+$ for $\Rightarrow$'s transitive closure, and $\Rightarrow^*$ for its reflexive and transitive closure.

Example: let us derive $E \Rightarrow^+ 42 + 35 * 17$

$$\begin{array}{ll} & E \\ \Rightarrow & E + E \\ \Rightarrow & 42 + E \\ \Rightarrow & 42 + E * E \\ \Rightarrow & 42 + 35 * E \\ \Rightarrow & 42 + 35 * 17 \end{array}$$

leftmost derivation

$$\begin{array}{ll} & E \\ \Rightarrow & E * E \\ \Rightarrow & E * 17 \\ \Rightarrow & E + E * 17 \\ \Rightarrow & E + 35 * 17 \\ \Rightarrow & 42 + 35 * 17 \end{array}$$

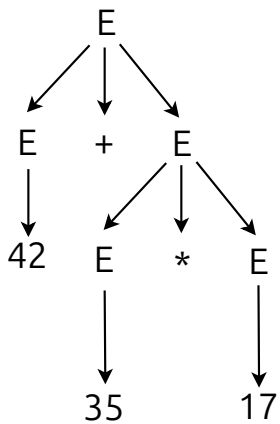
rightmost derivation

The **language** specified by a grammar: G is $\{w \in T^* \mid S \Rightarrow^+ w\}$

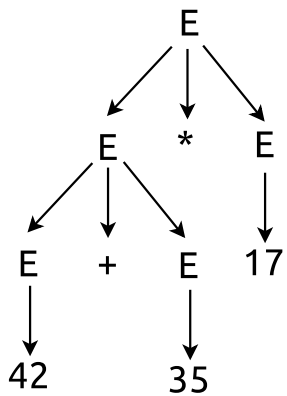
Parse Trees

Derivations can also be represented graphically as **parse trees**, that filter out the order in which productions are applied to replace non-terminals

$$
\begin{aligned}
& E \\
\Rightarrow\ & E + E \\
\Rightarrow\ & 42 + E \\
\Rightarrow\ & 42 + E * E \\
\Rightarrow\ & 42 + 35 * E \\
\Rightarrow\ & 42 + 35 * 17
\end{aligned}
$$



$$
\begin{aligned}
& E \\
\Rightarrow\ & E * E \\
\Rightarrow\ & E * 17 \\
\Rightarrow\ & E + E * 17 \\
\Rightarrow\ & E + 35 * 17 \\
\Rightarrow\ & 42 + 35 * 17
\end{aligned}
$$



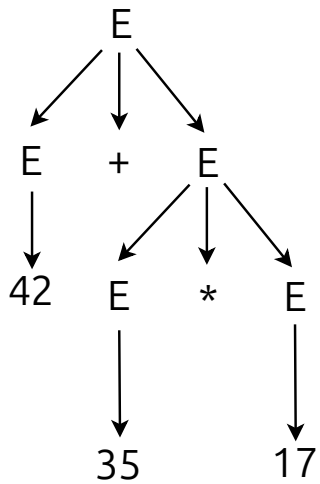
Find other derivations corresponding to those parse trees

For example, for the left one:

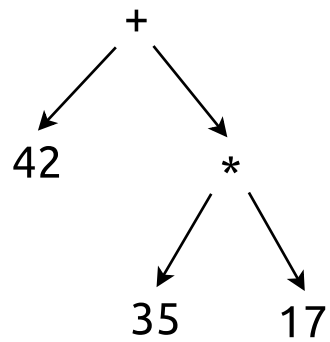
- $E \Rightarrow E + E \Rightarrow 42 + E \Rightarrow 42 + E * E \Rightarrow 42 + E * 17 \Rightarrow 42 + 35 * 17$

Concrete vs. Abstract Syntax Trees

Concrete syntax tree:



Abstract syntax tree:



Parsers build ASTs

Ambiguities

A language is **ambiguous** if a sentence can be derived with two different parse trees.

The following grammar is therefore ambiguous:

$$E ::= E + E \mid E * E \mid (E) \mid num$$

Ambiguous grammars are problematic. We cannot know which parse tree to select, and different parse trees might have different meaning.

Fact: Checking for ambiguity in CFGs is **undecidable**

However, often grammars can be **modified** to avoid ambiguities.

Fact: there are some languages that have ambiguous grammars but no unambiguous grammars.

Ambiguities

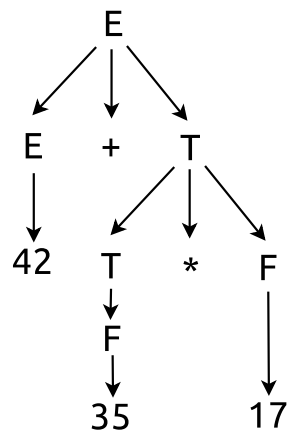
Example of an ambiguous grammar:

$$E ::= E + E \mid E * E \mid (E) \mid num$$

Unambiguous grammar:

$$\begin{array}{lcl} E & ::= & E + T \mid T \\ T & ::= & T * F \mid F \\ F & ::= & (E) \mid num \end{array}$$

The only parse tree for $42 + 35 * 17$ is now:



Pushdown Automata

What is the language specified by the grammar $E ::= aEb \mid \epsilon$?

The language is $\{a^n b^n \mid n \geqslant 0\}$.

Therefore CFGs can capture languages that REs cannot

FA were enough to recognize the languages corresponding to **REs**.

CFGs are accepted by **Pushdown Automata** (PDA), i.e., FA with stacks.

A PDA is defined as follows:

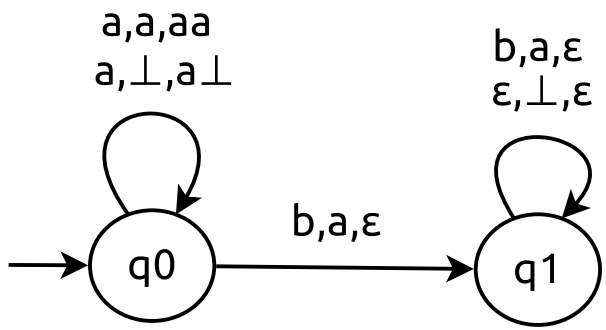
$$P = \langle Q, \Sigma, \Gamma, \delta, q_0, Z \rangle$$

where

- Q: a set of states
- Σ: an alphabet
- Γ: a set of stack symbols
- δ: a transition relation (i.e., $\forall q \in Q. \forall a \in \Sigma \cup \{\epsilon\}. \forall X \in \Gamma. \delta(q, a, X) \subseteq Q \times \Gamma^*$)
- q_0: a start state (i.e., $q_0 \in Q$)
- Z: an initial stack symbol (i.e., $Z \in \Gamma$)

Pushdown Automata

Example:



where: $Q = \{q_0, q_1\}$; $\Sigma = \{a, b\}$; $\Gamma = \{a, \bot\}$; $Z = \bot$

We draw a transition from q to q' labeled with a, X, α if $(q', \alpha) \in \delta(q, a, X)$

This means: when the PDA is in state q and reads a with X on top of the stack, it can move to state q' and replace X with α.

No final states: a word is accepted once we have read it and the stack is empty

Pushdown Automata

A word w is **accepted** by a PDA if there is a path from the start state labeled by w such that the final stack is empty.

An **instance description** (ID) is a triple (q, w, α), where $q \in Q$ is the current state, $w \in \Sigma^*$ the word to read, $\alpha \in \Gamma^*$ the current stack.

We can then define a reduction relation on IDs:

- $(q, aw, X\beta) \to (q', w, \alpha\beta)$ if $(q', \alpha) \in \delta(q, a, X)$
- $(q, w, X\beta) \to (q', w, \alpha\beta)$ if $(q', \alpha) \in \delta(q, \epsilon, X)$

The language accepted by a PDA $P = \langle Q, \Sigma, \Gamma, \delta, q_0, Z \rangle$ is:

$$\{w \in \Sigma^* \mid \exists q \in Q.(q_0, w, Z) \to^+ (q, \epsilon, \epsilon)\}$$

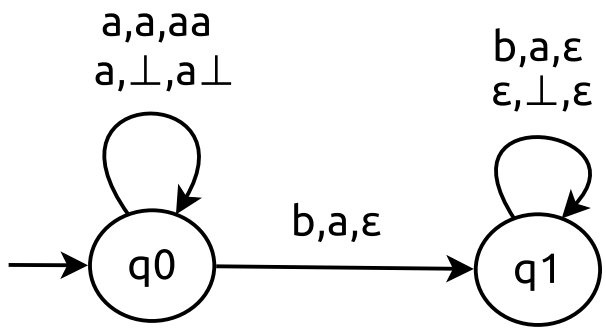
where $\to^+$ is the transitive closure of $\to$

Facts:

- for every CFG, there exists a PDA that accepts the same language
- for every PDA, there exists a CFG that accepts the same language

Pushdown Automata

Is $aabb$ accepted by?



Let us reduce $(q_0, aabb, \perp)$:

- $\rightarrow (q_0, abb, a\perp)$
- $\rightarrow (q_0, bb, aa\perp)$
- $\rightarrow (q_1, b, a\perp)$
- $\rightarrow (q_1, \epsilon, \perp)$
- $\rightarrow (q_1, \epsilon, \epsilon)$

Yes it is accepted

Parsers

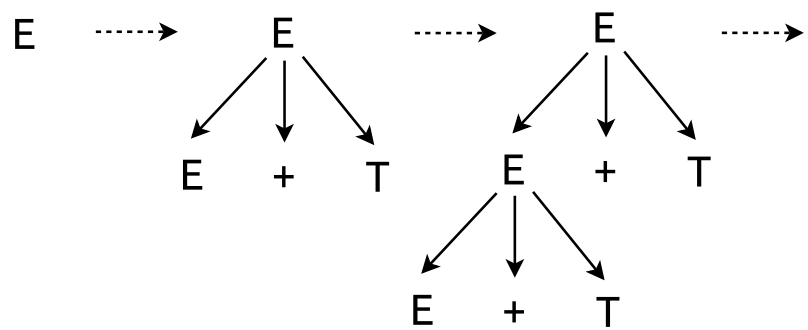
2 kinds of parsers:

- **top-down** (recursive descent): constructs the parse tree from the root (finds a leftmost derivation)
- **bottom-up**: constructs the parse tree from the leaves (finds a rightmost derivation)

Can we build a top-down parser for our grammar?

$$\begin{array}{lll} E & ::= & E + T \mid T \\ T & ::= & T * F \mid F \\ F & ::= & (E) \mid num \end{array}$$

The **left recursion** in $E ::= E + T \mid T$ leads to an infinite loop:



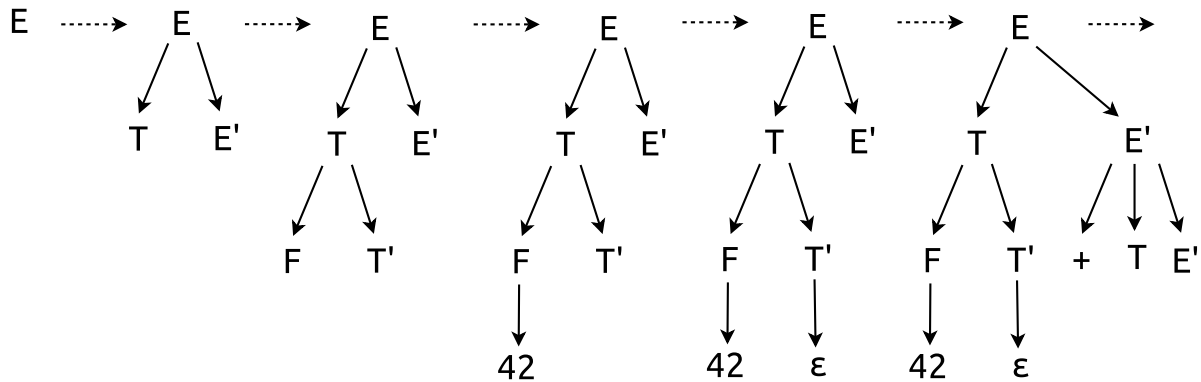
Top-Down Parsing

We must **eliminate left recursions**.

A general technique to eliminate **immediate left recursions** transforms the above grammar into (see Section 4.3.3 of the Dragon book):

$$
\begin{array}{lcl}
E & ::= & TE' \\
E' & ::= & +TE' \mid \epsilon \\
T & ::= & FT' \\
T' & ::= & *FT' \mid \epsilon \\
F & ::= & (E) \mid num
\end{array}
$$

We can now build a top-down parser that parses $42 + 35 * 17$ correctly – we allow ourselves to look ahead at the first symbol of the input:



Top-Down Parsing

Some non-terminals have multiple rules such as $E' ::= +TE' \mid \epsilon$, introducing some **non-determinism**: we have to choose which rule to apply.

In the above example, we could select the correct rule to apply because we allowed ourselves to look ahead at the first symbol of the input string.

It works here because **the grammar is LL(1)** (left-to-right parse, leftmost-derivation, 1-symbol lookahead), i.e., 1 symbol is enough to deterministically choose which production rule to use.

LL(1) grammars allow specifying most programming constructs. Although, writing such grammars is an art rather than a science.

In general top-down parsing may require looking ahead at more symbols (LL(k) – k-symbol lookahead) or backtracking if we have selected the wrong rule at some point (inefficient).

Top-Down Parsing

A simple **predictive parser** (first symbol is enough to choose which production to use) for the above grammar does a simple **recursive descent** and has 1 function for each non-terminal:

```
1: E() = T(); E'()
2: E'() = if (token() = "+") then eat("+"); T(); E'()
3: T() = F(); T'()
4: T'() = if (token() = "*") then eat("*"); F(); T'()
5: F() = if (token() = "(") then eat("("); E(); eat(")") else eat(NUM)
```

How can this be turned into a PDA? This is a PDA where the stack is the call stack (Sec. 4.4.4 in the Dragon book):

- **push**: E()
- **pop**: E(); **read**: ϵ; **push**: T(); E'()
- **pop**: E'(); **read**: +; **push**: eat("+"); T(); E'()
- **pop**: eat("+"); **read**: ϵ; **push**: ϵ
- ...

Bottom-Up Parsing

Issues with LL(k):

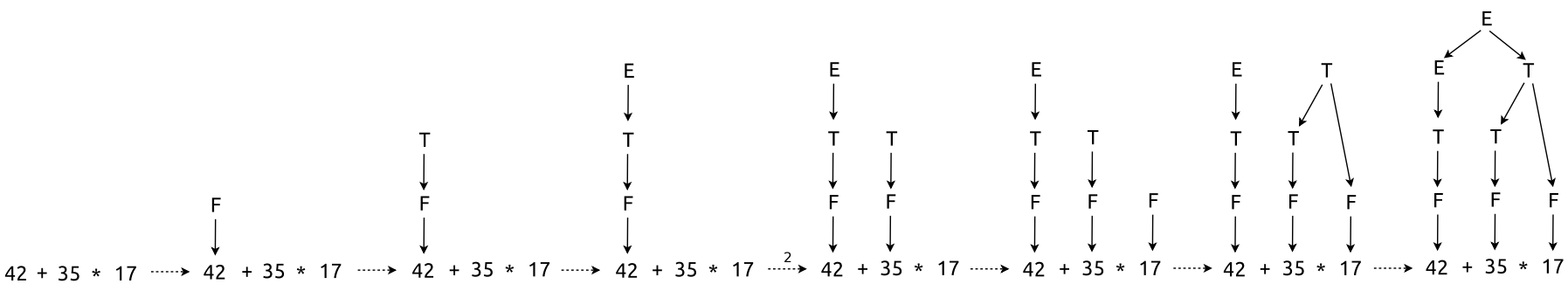
- must choose a derivation rule based on k tokens only
- requires eliminating left recursions
- some grammars are not LL(1)

Bottom-up parsing builds a parse tree from the leaves up to the root.

Example – Grammar (back to our left recursive):

$$\begin{array}{lcl} E & ::= & E + T \mid T \\ T & ::= & T * F \mid F \\ F & ::= & (E) \mid num \end{array}$$

A rightmost derivation (reversed) of a parse tree for $42 + 35 * 17$?:



Bottom-Up Parsing

In particular LR(k) (or shift-reduce) parsers work as follows

Initially:

- input string to parse: $w\$$
- stack: $\$$ (start symbol)

High-level algorithm:

- **shift** input symbols onto the stack
- **reduce** a string β of grammar symbols on top of the stack to the head of the appropriate production rule
- **repeat** until the stack contains the start symbol and the input is empty, or an error is detected
- allowed to **look ahead** at k symbols to make a decision (in practice $k \leqslant 1$)

Fact: LR parsers can parse more grammars than LL parsers

Bottom-Up Parsing

Execute this algorithm on 42+35*17

stack	input	action
$\$$	$42 + 35 * 17\$$	shift
$\$\ 42$	$+35 * 17\$$	reduce using $F ::= num$
$\$\ F$	$+35 * 17\$$	reduce using $T ::= F$
$\$\ T$	$+35 * 17\$$	reduce using $E ::= T$
$\$\ E$	$+35 * 17\$$	shift
$\$\ E\ +$	$35 * 17\$$	shift
$\$\ E\ +\ 35$	$*17\$$	reduce using $F ::= num$
$\$\ E\ +\ F$	$*17\$$	reduce using $T ::= F$
$\$\ E\ +\ T$	$*17\$$	shift
$\$\ E\ +\ T\ *$	$17\$$	shift
$\$\ E\ +\ T\ *\ 17$	$\$$	reduce using $F ::= num$
$\$\ E\ +\ T\ *\ F$	$\$$	reduce using $T ::= T * F$
$\$\ E\ +\ T$	$\$$	reduce using $E ::= E + T$
$\$\ E$	$\$$	accept

Bottom-Up Parsing

How do we know whether to shift or reduce?

In particular, in the highlighted line above, why didn't we reduce using $E ::= T$ or $E ::= E + T$?

LR parsers use **items**: they make decisions by keeping track of where they are in a parse in items.

An LR(0) item is a production rule with a dot in the right-hand-side, indicating how much of the production has already been seen.

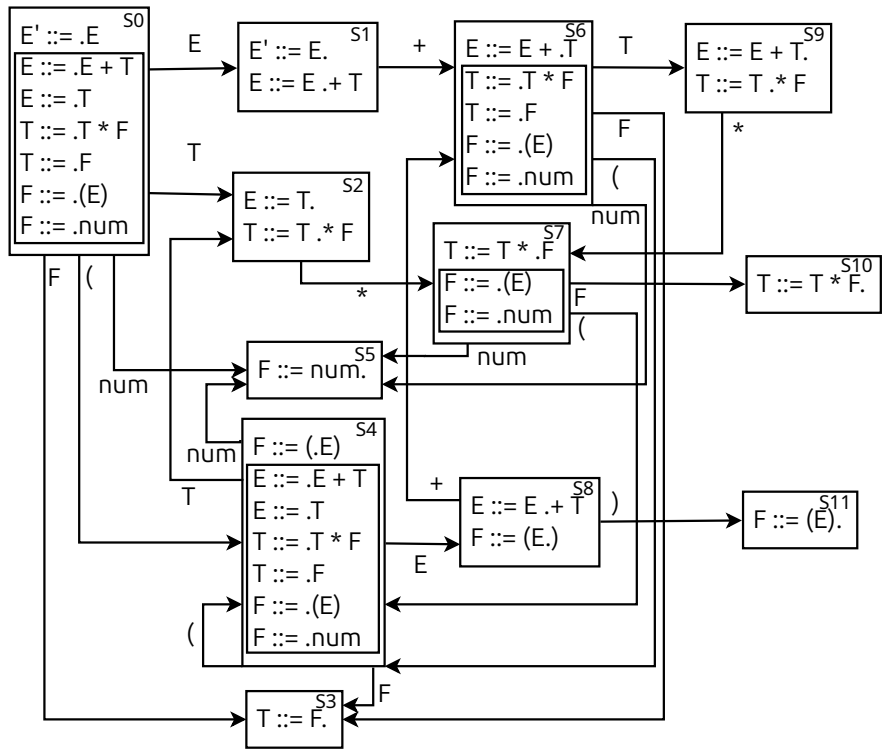
First, we introduce a new "initial" production rule: $E' ::= E$ (indicates when to stop parsing)

The items of the above grammar are:

$E' ::= \cdot E$	$E' ::= E\cdot$		
$E ::= \cdot E + T$	$E ::= E \cdot + T$	$E ::= E + \cdot T$	$E ::= E + T\cdot$
$E ::= \cdot T$	$E ::= T\cdot$		
$T ::= \cdot T * F$	$T ::= T \cdot * F$	$T ::= T * \cdot F$	$T ::= T * F\cdot$
$T ::= \cdot F$	$T ::= F\cdot$		
$F ::= \cdot (E)$	$F ::= (\cdot E)$	$F ::= (E\cdot)$	$F ::= (E)\cdot$
$F ::= \cdot num$	$T ::= num\cdot$		

Bottom-Up Parsing

We build the following LR(0) automaton:



- start with $I_0 = \{E' ::= \cdot E\}$
- compute $S_0 = \bigcup_{n\in\mathbb{N}} \texttt{closure}^n(I_0)$:
  $\texttt{closure}(I) = \{B ::= .\gamma \mid A ::= \alpha \cdot B\beta \in I \wedge B ::= .\gamma \text{ is an item}\}$
- add a transition labeled B from S_0 to $\bigcup_{n\in\mathbb{N}} \texttt{closure}^n(I)$, where $I = \{A ::= \alpha B \cdot \beta \mid A ::= \alpha \cdot B\beta \in S_0\}$
- repeat

Bottom-Up Parsing

Simple LR (SLR) uses of an LR(0) automaton as follows (in that order):

- if the next symbol is a, and in a state containing $A ::= \alpha \cdot a\beta$, then shift a and move to the next state as indicated by the automaton.
- if the next symbol is a, and in a state containing $A ::= \alpha\cdot$, such that it is possible to derive a string where a follows A, then reduce using $A ::= \alpha$, go back to a previous state according to α, and follow the A transition.

Going back to our example, we start in S_0:

- stack \$, input $42 + 35 * 17\$$: shift 42, transition $S_0 \to S_5$
- stack \$ 42, input $+35 * 17\$$: item $F ::= num\cdot$, $F+$ derivable, reduce using $F ::= num$, read F, transition $S_5 \to S_0 \to S_3$
- stack \$ F, input $+35 * 17\$$: item $T ::= F\cdot$, $T+$ derivable, reduce using $T ::= F$, read T, transition $S_3 \to S_0 \to S_2$
- stack \$ T, input $+35 * 17\$$: cannot read $*$, item $E ::= T\cdot$, $E+$ derivable, reduce using $E ::= T$. read E, transition $S_2 \to S_0 \to S_1$
- stack \$ E, input $+35 * 17\$$: shift $+$, transition $S_1 \to S_6$
- stack \$ E $+$, input $35 * 17\$$: shift 35, transition $S_6 \to S_5$
- stack \$ E $+$ 35, input $*17\$$: item $F ::= num\cdot$, $F*$ derivable, reduce using $F ::= num$, read F, transition $S_5 \to S_6 \to S_3$
- stack \$ E $+$ F, input $*17\$$: item $T ::= F\cdot$, $T*$ derivable, reduce using $T ::= F$, read T, transition $S_3 \to S_6 \to S_9$
- stack \$ E $+$ T, input $*17\$$: **shift** $*$, transition $S_9 \to S_7$

Bottom-Up Parsing

There may still be shift/reduce or reduce/reduce conflicts.

Conclusion

What did we cover today?

- Context-Free Grammars
- Top-down parsing
- Bottom-up parsing

Next time?

- Semantic analysis